

Software Design Specification

Air Ticket Reservation System

Project Code: ATRS-2025-SDD
Internal Advisor:
External Advisor:
Project Manager: Muhammad Ehtisham
Project Team: Muhammad Ehtisham, Malik
Muhammad Umair, Ali Zain
Submission Date:

Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	Air Ticket Reservation System
Document	Software Design Specification
Document Version	1.0
Identifier	ATRS-2025-SDD
Status	Final Draft
Author(s)	Muhammad Ehtisham, Malik Muhammad Umair, Ali Zain
Approver(s)	Project Manager / Internal Advisor
Issue Date	2025
Document Location	Project Repository / Documentation Folder
Distribution	1. Internal Advisor 2. Project Manager 3. Project Team

Definition of Terms, Acronyms and Abbreviations

This section provides definitions of all terms, acronyms, and abbreviations required to correctly interpret this document.

Term / Acronym	Definition
ATRS	Air Ticket Reservation System
SDD	Software Design Document
SRS	Software Requirements Specifications
OOP	Object-Oriented Programming
STL	C++ Standard Template Library
CLI	Command Line Interface
DFD	Data Flow Diagram
ER	Entity-Relationship (diagram)
UML	Unified Modelling Language
CNIC	Computerized National Identity Card
DIA	Dubai International Airport
SIA	Sharjah International Airport
AIA	Abu Dhabi International Airport / King Abdulaziz International Airport
KIA	King Khalid International Airport, Riyadh
FIA	King Fahd International Airport, Dammam
LHE	IATA code – Allama Iqbal International Airport, Lahore
ISB	IATA code – New Islamabad International Airport
FSD	IATA code – Faisalabad International Airport
PKR	Pakistani Rupee – base currency for pricing
AED	UAE Dirham
SAR	Saudi Riyal
PM	Project Manager
DS	Design Specification

Table of Contents

1. Introduction	5
1.1 Purpose of Document	5
1.2 Project Overview	5
1.3 Scope	5
2. Design Considerations	6
2.1 Assumptions and Dependencies	6
2.2 Risks and Volatile Areas	6
3. System Architecture	7
3.1 System Level Architecture	7
3.2 Sub-System / Component / Module Level Architecture	8
3.3 Sub-Component / Sub-Module Level Architecture	9
4. Design Strategies	10
4.1 Modular Object-Oriented Design Strategy	10
4.2 Data Management Strategy	10
4.3 User Interface Strategy	10
4.4 Error Handling and Robustness Strategy	10
4.5 Security Strategy	11
5. Detailed System Design	11
5.1 Class Diagram	11
5.2 Data Flow Diagrams (DFD)	14
5.3 Use Case Diagram	18
5.4 Entity-Relationship (ER) Diagram	19
5.5 Sequence Diagrams	20
5.6 State Transition Diagram	22
6. References	23
7. Appendices	23

1. Introduction

1.1 Purpose of Document

This Software Design Specification (SDD) document describes the complete architectural and detailed design of the Air Ticket Reservation System (ATRS). The purpose of this document is to explain how the system is built based on the functional and non-functional requirements defined in the Software Requirements Specification (ATRS-2025-SRS v4.0).

This document employs an Object-Oriented Design (OOD) methodology that maps directly to the C++ class hierarchy implemented in the source code. All design decisions presented here are validated against the actual implementation to guarantee accuracy and traceability.

Intended audience for this document:

- Software Developers
- System Designers and Architects
- QA Engineers and Testers
- Project Manager and Internal / External Advisors

1.2 Project Overview

The Air Ticket Reservation System (ATRS) is a console-based, object-oriented C++ application that enables passengers to search, book, and manage air ticket reservations for flights departing from Pakistani airports to international destinations. The system supports the complete reservation lifecycle: from user account creation and flight discovery, through seat selection and payment processing, to receipt generation and booking cancellation.

The system is implemented as a single-process, single-threaded CLI application using C++ Standard Library (STL) containers for all in-memory data management. It follows a layered OOP inheritance hierarchy: Person → User_authentication → Booking → Payment → Receipt, with supporting classes for Flights, Search_flight, and Seat_management.

Key capabilities include:

- Multi-passenger booking (up to 5 passengers per transaction)
- Multi-currency payment processing (5 currencies with PKR conversion)
- Real-time in-session seat availability tracking
- Input validation and graceful error recovery throughout

1.3 Scope

The system is scoped to the following capabilities:

IN SCOPE:

- User account management: registration, login, profile viewing, profile editing, and password recovery
- Flight search by departure airport (Lahore, Faisalabad, Islamabad), destination country (UAE, Saudi Arabia), airport code, trip type, and travel date

- Booking process: passenger details, class selection (Economy / Business / First Class), seat selection, baggage specification, and fare computation
- Payment processing: debit card, credit card, and online banking; five currency options with automatic PKR conversion
- Receipt generation displaying all passenger and flight details
- Flight cancellation by CNIC with seat release
- Comprehensive input validation and error handling

OUT OF SCOPE:

- Persistent data storage – all data is in-memory only
- Graphical user interface
- Real-time airline API integration
- Actual OTP / SMS delivery and real payment gateway integration
- Concurrent multi-user access
- Email notifications or network-based communication
- Destinations other than UAE and Saudi Arabia

2. Design Considerations

This section addresses the issues that must be resolved before devising the complete design solution. It establishes the groundwork for the system design by examining assumptions, dependencies, and risks from a design perspective.

2.1 Assumptions and Dependencies

The following assumptions and dependencies underpin the ATRS design:

ID	Type	Description
DA-01	Assumption	All user input is in English; no multi-language support is required
DA-02	Assumption	The host system clock is correctly set to local time (GMT+5, Pakistan Standard Time); date validation relies on <code>std::chrono::system_clock</code>
DA-03	Assumption	The system runs in a single-process, single-user mode with no concurrent access
DA-04	Assumption	Flight base prices (UAE_PRICE = PKR 100,000; SA_PRICE = PKR 150,000) are compile-time constants; price changes require recompilation
DA-05	Assumption	Currency exchange rates (AED=76.37, EUR=310, GBP=360, SAR=74.79, USD=285) are fixed at development time
DA-06	Assumption	The OTP in password recovery is simulated; any 4-character string is accepted
DA-07	Assumption	Valid flight dates are restricted to 2025–2026 (hardcoded constraint)
DD-01	Dependency	The C++ Standard Library (STL) version compliant with ISO C++11 or later
DD-02	Dependency	Host OS providing POSIX-compatible time functions (<code>std::localtime</code> , <code>std::chrono::system_clock</code>)
DD-03	Dependency	g++ / Clang / MSVC compiler supporting C++17 standard

2.2 Risks and Volatile Areas

The following risks and volatile areas may impact the ATRS design:

Risk ID	Volatile Area	Impact	Design Mitigation
R-01	In-memory data loss on crash	High – all session data is lost	Modular Booking/Payment classes allow future file-I/O adapters to be added without altering core logic
R-02	Hardcoded flight dataset	Medium – adding new	Flight data is isolated in the Flights class; new methods can be added independently

Risk ID	Volatile Area	Impact	Design Mitigation
		routes requires source changes	
R-03	Hardcoded date range (2025–2026)	Medium – system becomes invalid after 2026	check_valid_date() is a single function; the year range can be updated in one location
R-04	No real payment validation	Medium – any amount is accepted after conversion	Payment logic is isolated in the Payment class; a real gateway can replace payment_checker()
R-05	Single-user concurrency model	Low – current scope is single-user	Static STL containers can be replaced with thread-safe structures in a future multi-user version
R-06	Password stored in plain text in memory	Medium – security concern for future web deployment	Passwords are in volatile memory only; a hashing mechanism can be introduced in the User_authentication class

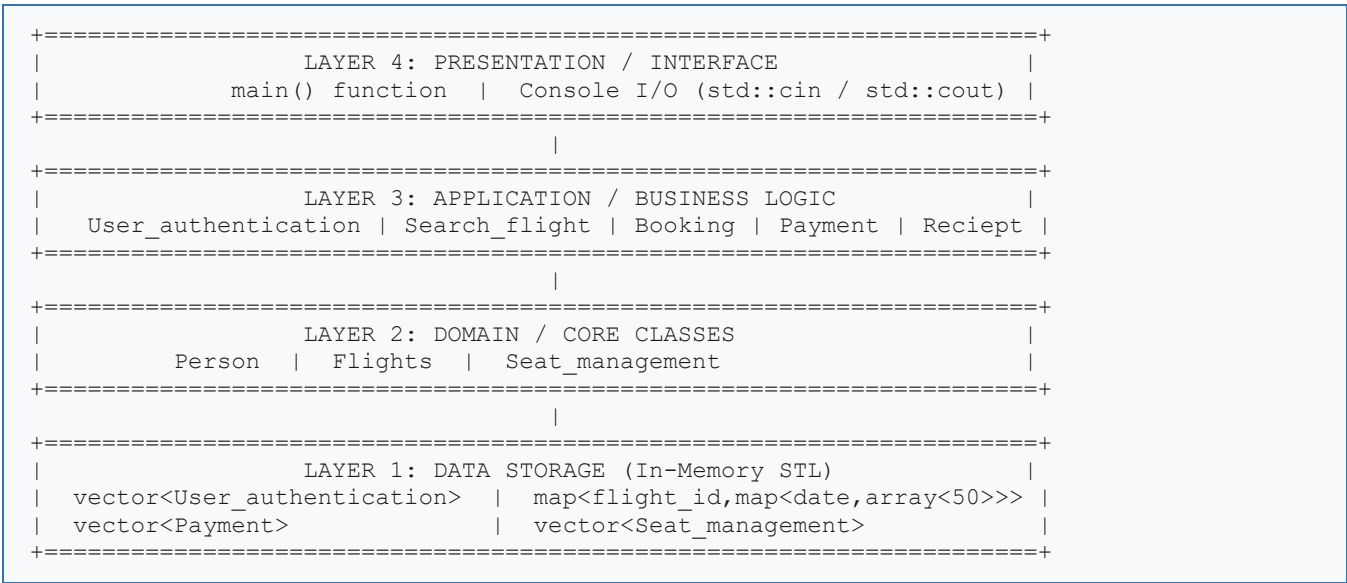
3. System Architecture

This section provides a high-level overview of how the functionality and responsibilities of the ATRS are partitioned and assigned to subsystems and components. The primary architectural style is a layered Object-Oriented architecture implemented as a single-process C++ console application.

3.1 System Level Architecture

The ATRS follows a Layered Object-Oriented Architecture. The system is partitioned into four logical layers that correspond directly to the C++ class hierarchy and the functional modules identified in the SRS.

Figure 3.1 – System Level Architecture (Layered View)



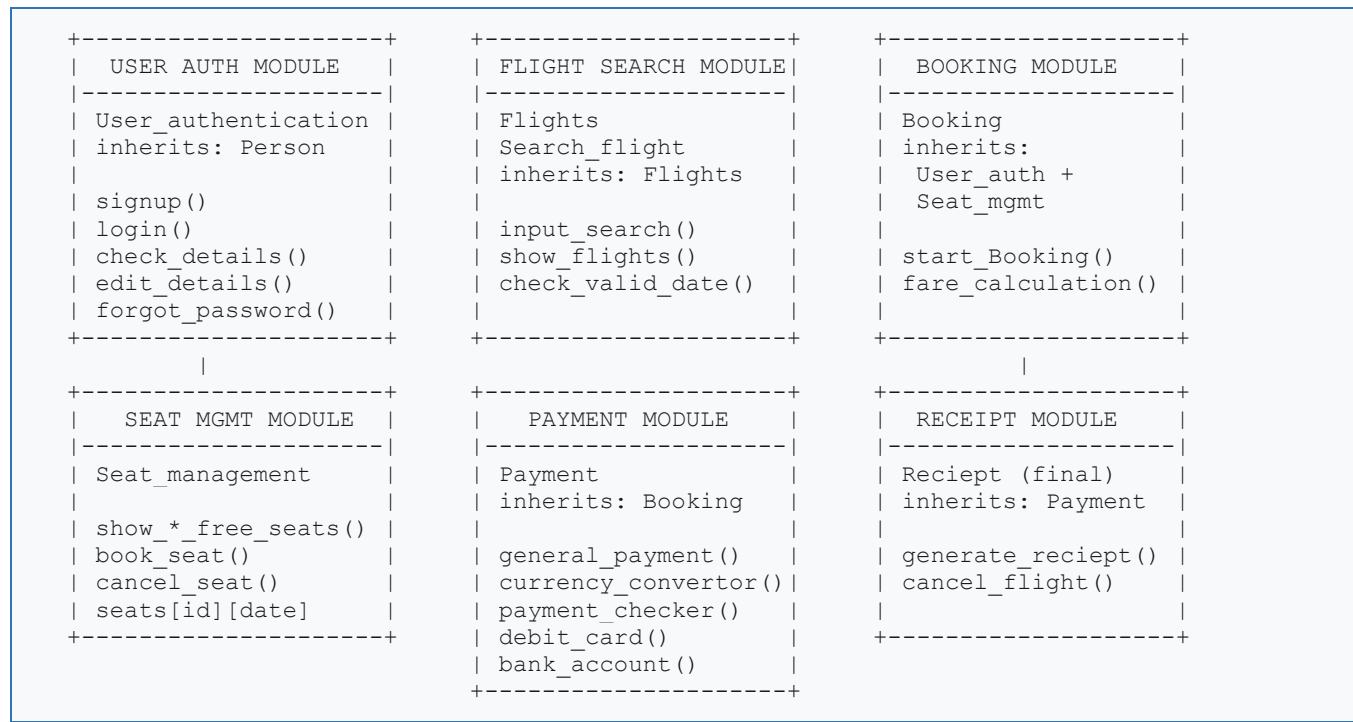
Layer responsibilities:

Layer	Components	Responsibility
Layer 4 – Presentation	main(), std::cin/cout	Presents the main menu, routes user choices to the appropriate business-logic objects, and renders results to the console
Layer 3 – Business Logic	User_authentication, Search_flight, Booking, Payment, Reciept	Implements all functional requirements: authentication, flight search, booking workflow, payment, receipt, and cancellation
Layer 2 – Domain Classes	Person, Flights, Seat_management	Provides the core domain abstractions: personal data validation, static flight data, and seat availability management
Layer 1 – Data Storage	STL containers (vector, map, array)	Maintains all in-session state: registered users, booked payments, seat maps keyed by flight ID and date

3.2 Sub-System / Component / Module Level Architecture

The ATRS is decomposed into six functional modules. Each module corresponds to one or more C++ classes and directly maps to a group of SRS functional requirements.

Figure 3.2 – Module-Level Component Architecture



Module	Primary Class(es)	SRS Requirements Covered
User Authentication	User_authentication (inherits Person)	FR-UA-01 to FR-UA-05
Flight Search	Flights, Search_flight	FR-FS-01, FR-FS-02
Seat Management	Seat_management	FR-BP-03
Booking	Booking (inherits User_authentication + Seat_management)	FR-BP-01 to FR-BP-05
Payment	Payment (inherits Booking)	FR-PM-01 to FR-PM-03
Receipt & Cancellation	Reciept / final (inherits Payment)	FR-RC-01, FR-CF-01

3.3 Sub-Component / Sub-Module Level Architecture

Each module is further decomposed into sub-components corresponding to concrete methods and data structures in the implementation.

Figure 3.3a – User Authentication Sub-Component Decomposition

```

User_authentication (inherits Person)
|
+-- [Data]   email: string
|            password: string
|            check_signup: bool
|            static user: vector<User_authentication>
|
+-- [Input Validation Sub-Component]
|   check_valid_name() <-- shared free function
|
+-- [Registration Sub-Component]
|   signup() --> person_input() --> password_validation_loop()
|
+-- [Authentication Sub-Component]
|   login() --> credential_search_loop(user vector)
|
+-- [Profile Management Sub-Component]
|   check_details() | edit_details() | forgot_password()

```

Figure 3.3b – Booking Sub-Component Decomposition

```

Booking (inherits User_authentication + Seat_management)
|
+-- [Passenger Sub-Component]
|   Collects no_of_passengers, adult/child/infant counts
|   Loops Person::person_input() per passenger
|
+-- [Class Selection Sub-Component]
|   Class choice (1=Economy, 2=Business, 3=First Class)
|   Routes to show_*_free_seats() then book_seat()
|
+-- [Baggage & Fare Sub-Component]
|   Accepts weight [0..20 kg], calls fare_calculation()
|   Fare = Base_Price + (weight * 1000 PKR)

```

Figure 3.3c – Seat Management Sub-Component Decomposition

```

Seat_management
|
+-- [Static Seat Map]
|   map<flight_id, map<date_string, array<int,50>>>
|   Seats 1-30: Economy | 31-40: Business | 41-50: First Class
|
+-- [Availability Display]
|   show_seats(class_choice, id, date)
|   Displays only seats with value 0 (available)
|
+-- [Booking Sub-Component]
|   book_seat(n, id, date) --> sets seat[idx]=1
|
+-- [Cancellation Sub-Component]
|   cancel_seat(id, date) --> sets seat[idx]=0, removes from booked_seat vector

```

4. Design Strategies

This section describes the key design strategies and decisions that shape the overall organization of the ATRS. Each strategy provides reasoning, trade-offs, and alignment with the project goals.

4.1 Modular Object-Oriented Design Strategy

The ATRS adopts a strict OOP inheritance hierarchy as its primary structural strategy. The hierarchy (Person → User_authentication → Booking → Payment → Reciept) provides natural code reuse: shared attributes and methods are defined once in a base class and inherited by all derived classes. Multiple inheritance (Booking inherits both User_authentication and Seat_management) allows the booking workflow to compose authentication and seat-management capabilities without duplication.

Trade-off: Deep inheritance chains increase coupling between layers. Changes to a base class (e.g., Person) propagate to all derived classes. This is acceptable for the current single-developer academic context but would require refactoring to composition-over-inheritance for a production system.

4.2 Data Management Strategy

All runtime data is held in volatile in-memory STL containers:

- `vector<User_authentication>` stores all registered user accounts
- `vector<Payment>` stores all confirmed bookings
- `map<string, map<string, array<int,50>>>` stores seat availability keyed by flight ID and date string

The use of static class members ensures that a single shared instance of each container is accessible across all objects in the session. This avoids passing containers as parameters and mimics a simple in-memory database.

Trade-off: All data is lost on program termination. A future version should introduce file I/O or SQLite persistence, which can be added by extending the data-access layer without modifying business logic classes.

4.3 User Interface Strategy

The ATRS uses a numbered main-menu CLI paradigm. All interaction flows are prompt-driven with do-while loops that re-prompt on invalid input rather than crashing. This design ensures robustness: the system never terminates abnormally due to user input errors.

Every input-gathering function clears `std::cin` and ignores buffered characters on non-numeric input failure, following a consistent error-recovery pattern throughout the codebase. All prompts specify the expected input format (e.g., "Enter Departure Date (x/y/z) = ").

4.4 Error Handling and Robustness Strategy

The system implements a defensive programming strategy across all input points:

- Numeric fields use `cin.fail()` detection followed by `cin.clear() + cin.ignore(1000, '\n')`
- Name fields use the `check_valid_name()` free function to reject digits and listed special characters
- Date fields use `check_valid_date()` which validates year range, month/day bounds, and past-date rejection against the system clock
- Airport and country codes are converted to lowercase before comparison to ensure case-insensitive matching
- Seat selection validates class-range membership and availability before committing

4.5 Security Strategy

Within the constraints of an in-memory CLI application, the following security principles are applied:

- Authentication is required before any booking operation can proceed
- Passwords must satisfy a minimum complexity policy (8+ chars, upper, lower, digit, punctuation)
- Password recovery requires both email and phone number verification
- Flight cancellation verifies CNIC ownership before releasing a seat
- No credentials or personal data are written to disk or transmitted over a network

5. Detailed System Design

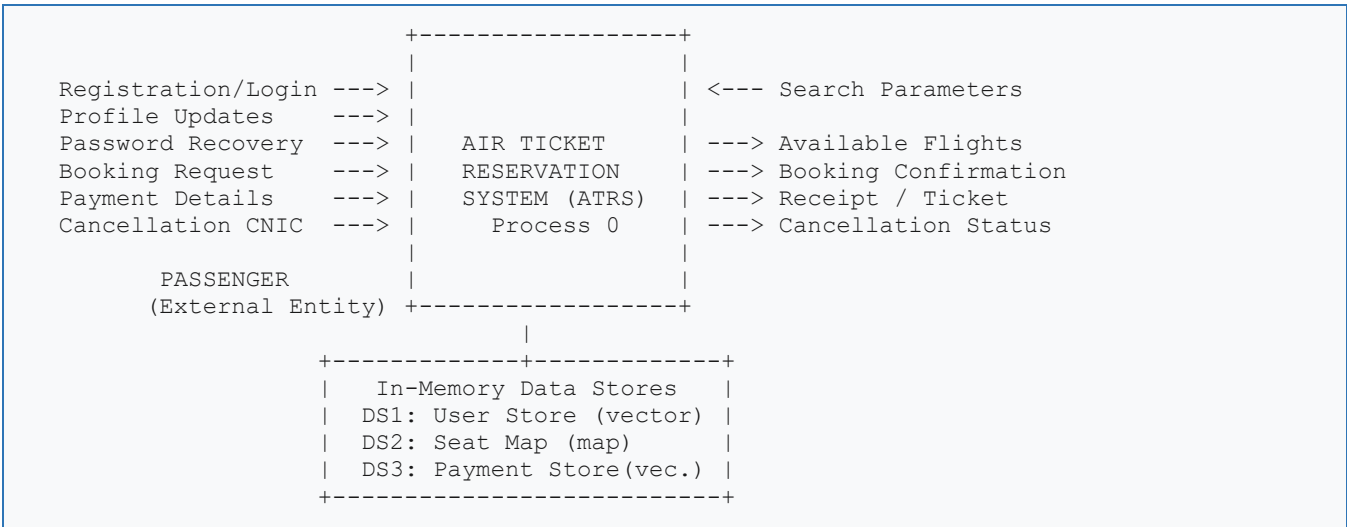
This section provides the complete detailed design artefacts for the ATRS, Data Flow Diagrams (DFD Level 0), Use Case diagram.

5.1 Data Flow Diagrams (DFD)

5.1.1 DFD Level 0 – Context Diagram

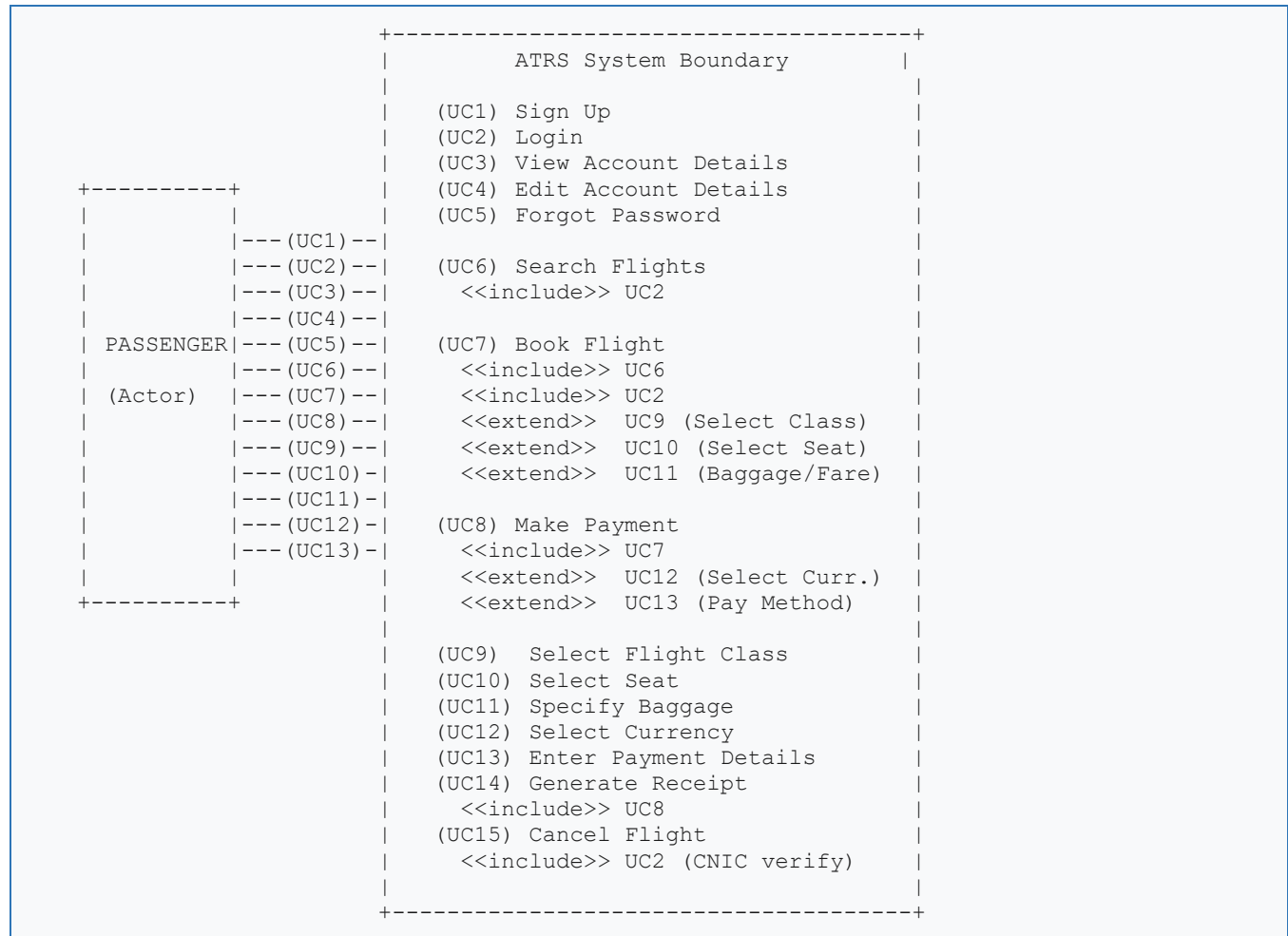
The Level 0 DFD (Context Diagram) represents the ATRS as a single process (black box) showing all external entities and the high-level data flows between them.

Figure 5.1.1 – DFD Level 0: Context Diagram



5.2 Use Case Diagram

The following Use Case Diagram identifies the single actor (Passenger) and all major use cases of the ATRS. Relationships (include and extend) are derived from the SRS and validated against the source code flow.

Figure 5.2 – ATRS Use Case Diagram

Use Case ID	Name	Actor	Relationship	SRS Req.
UC1	Sign Up	Passenger	—	FR-UA-01
UC2	Login	Passenger	—	FR-UA-02
UC3	View Account Details	Passenger	—	FR-UA-03
UC4	Edit Account Details	Passenger	—	FR-UA-04
UC5	Forgot Password	Passenger	—	FR-UA-05
UC6	Search Flights	Passenger	—	FR-FS-01, 02

Use Case ID	Name	Actor	Relationship	SRS Req.
UC7	Book Flight	Passenger	<<include>> UC6, UC2	FR-BP-01–05
UC8	Make Payment	Passenger	<<include>> UC7	FR-PM-01–03
UC9	Select Flight Class	Passenger	<<extend>> UC7	FR-BP-02
UC10	Select Seat	Passenger	<<extend>> UC7	FR-BP-03
UC11	Specify Baggage	Passenger	<<extend>> UC7	FR-BP-04
UC12	Select Currency	Passenger	<<extend>> UC8	FR-PM-01
UC13	Enter Payment Details	Passenger	<<extend>> UC8	FR-PM-02
UC14	Generate Receipt	Passenger	<<include>> UC8	FR-RC-01
UC15	Cancel Flight	Passenger	<<include>> CNIC verify	FR-CF-01

6. References

Ref. No.	Document Title	Date	Source
ATRS-2025-SRS-V4	Air Ticket Reservation System – SRS v4.0	2025	Project Repository / Documentation Folder
ATRS-2025-CODE	ATRS C++ Source Code (main.cpp)	2025	Project Repository
ATRS-2025-SRS-V1	Air Ticket Reservation System – SRS v1.0	2025	Project Documentation Folder
ATRS-2025-DS-V1	Design Document – Air Ticket Reservation System v1.0	2025	Project Documentation Folder
ATRS-2025-TMPL	Software Design Specification Template 2025	2025	FAST-NU / UOS Academic Repository
ISO-25010	ISO/IEC 25010 – Software Quality Model	2011	ISO Standards Body
IEEE-830	IEEE Std 830-1998 – Recommended Practice for SRS	1998	IEEE
ISO-CPP17	ISO/IEC 14882:2017 – C++17 Standard	2017	ISO Standards Body

7. Appendices

Appendix A – Complete Flight ID Reference

The following table lists all 36 unique flight IDs implemented in the ATRS (18 one-way + 18 return):

Departure	Country	Airport	Type	Flight Name	Flight ID
Lahore	UAE	DIA	One-Way	PIA Express	LHE-DXB-30T1R
Lahore	UAE	DIA	Return	PIA Gulf1	LHE-DXB-301R
Lahore	UAE	SIA	One-Way	AirBlue Sky	LHE-SIA-20E1F
Lahore	UAE	SIA	Return	PIA Gulf2	LHE-SIA-201F
Lahore	UAE	AIA	One-Way	New PIA	LHE-AIA-30Y1R
Lahore	UAE	AIA	Return	PIA Gulf3	LHE-AIA-101A
Lahore	Saudi Arabia	KIA	One-Way	PIA Gulf11	LHE-KIA-12B09F
Lahore	Saudi Arabia	KIA	Return	PIA Gulf11	LHE-KIA-409F
Lahore	Saudi Arabia	FIA	One-Way	PIA Gulf12	LHE-FIA-A2701F
Lahore	Saudi Arabia	FIA	Return	PIA Gulf12	LHE-FIA-1C01A
Lahore	Saudi Arabia	AIA	One-Way	PIA Gulf13	LHE-AIA-BF301R
Lahore	Saudi Arabia	AIA	Return	PIA Gulf13	LHE-AIA-30S1R
Faisalabad	UAE	DIA	One-Way	PIA Express	FIA-DXB-W6301R
Faisalabad	UAE	DIA	Return	PIA Gulf1	FIA-DXB-N3501R
Faisalabad	UAE	SIA	One-Way	AirBlue Sky	FIA-SIA-S201F
Faisalabad	UAE	SIA	Return	PIA Gulf2	FIA-SIA-2041FE
Faisalabad	UAE	AIA	One-Way	New PIA	FIA-AIA-M301R
Faisalabad	UAE	AIA	Return	PIA Gulf3	FIA-AIA-1011AD
Faisalabad	Saudi Arabia	KIA	One-Way	PIA Gulf11	FIA-KIA-12L09F
Faisalabad	Saudi Arabia	KIA	Return	PIA Gulf11	FIA-KIA-2409GF

Departure	Country	Airport	Type	Flight Name	Flight ID
Faisalabad	Saudi Arabia	FIA	One-Way	PIA Gulf12	FIA-FIA-27P01F
Faisalabad	Saudi Arabia	FIA	Return	PIA Gulf12	FIA-FIA-320Q1F
Faisalabad	Saudi Arabia	AIA	One-Way	PIA Gulf13	FIA-AIA-F301R
Faisalabad	Saudi Arabia	AIA	Return	PIA Gulf13	FIA-AIA-V3301R
Islamabad	UAE	DIA	One-Way	PIA Express	ISB-DXB-6301R0
Islamabad	UAE	DIA	Return	PIA Gulf1	ISB-DXB-3501RZ
Islamabad	UAE	SIA	One-Way	AirBlue Sky	ISB-SIA-X201F
Islamabad	UAE	SIA	Return	PIA Gulf2	ISB-SIA-2041FQ
Islamabad	UAE	AIA	One-Way	New PIA	ISB-AIA-J301R
Islamabad	UAE	AIA	Return	PIA Gulf3	ISB-AIA-1011AO
Islamabad	Saudi Arabia	KIA	One-Way	PIA Gulf11	ISB-KIA-1209F
Islamabad	Saudi Arabia	KIA	Return	PIA Gulf11	ISB-KIA-2409F
Islamabad	Saudi Arabia	FIA	One-Way	PIA Gulf12	ISB-FIA-2701F
Islamabad	Saudi Arabia	FIA	Return	PIA Gulf12	ISB-FIA-K3201F
Islamabad	Saudi Arabia	AIA	One-Way	PIA Gulf13	ISB-AIA-F301R
Islamabad	Saudi Arabia	AIA	Return	PIA Gulf13	ISB-AIA-D3301R

Appendix B – Currency Exchange Rates

Choice	Currency	Code	Rate (PKR per unit)
1	UAE Dirham	AED	76.37
2	Euro	EUR	310.00
3	British Pound	GBP	360.00
4	Saudi Riyal	SAR	74.79
5	US Dollar	USD	285.00

Appendix C – Seat Layout Reference

Seat Range	Class	Count	Input Range
1 – 30	Economy	30 seats	Enter 1–30
31 – 40	Business	10 seats	Enter 31–40
41 – 50	First Class	10 seats	Enter 41–50

Appendix D – Main Menu Options Reference

Option	Function	Implements
1	Sign Up	FR-UA-01
2	Login	FR-UA-02
3	Check Account Details	FR-UA-03
4	Edit Account Details	FR-UA-04
5	Forgot Password	FR-UA-05
6	Search Flight	FR-FS-01, FR-FS-02
7	Flight Booking	FR-BP-01 to FR-BP-05, FR-PM-01 to FR-PM-03, FR-RC-01
8	Cancel Flight	FR-CF-01
9	Exit	—

End of Software Design Specification Document